4.6 Интерфейсы

Задание общего поведения

Пакет **fmt** определяет интерфейс Stringer, который имеет метод `String() string`для предоставления строкового представления типа

```
type Stringer interface {
  String() string
}

func main() {
  t := time.Now()
  fmt.Println(t)
}
```

```go
package main

import "fmt"

type Person struct {
  Name string
  Age int
}

func (p Person) String() string {
  return fmt.Sprintf("%v (%v years)", p.Name, p.Age)
}

func main() {
  a := Person{"Arthur Dent", 42}
  z := Person{"Zaphod Beeblebrox", 101}
  fmt.Println(a, "|", z)
}
```

Уменьшение связности кода

```go
package main

import (
    "errors" "fmt"
)

type Customer struct { ID string
}

// Конкретная реализация хранилища на основе MySQL
type MySQLStore struct{}

// StoreCustomer сохраняет клиента в MySQL
func (m MySQLStore ) StoreCustomer (customer Customer ) error {
    fmt.Println ("Сохранение клиента в MySQL:" , customer .ID) return nil
}

// CustomerService зависит от конкретной реализации MySQLStore
type CustomerService struct {
    store MySQLStore
}

// CreateNewCustomer создает нового клиента и сохраняет его
func (cs CustomerService ) CreateNewCustomer (id string) error {
    customer := Customer{ID: id}
    return cs.store.StoreCustomer (customer)
}

func main() {
    service := CustomerService {store: MySQLStore {}}
    err := service.CreateNewCustomer ("123")
    if err != nil {
        fmt.Println ("Ошибка:", err)
    }
}
```

Создание абстракции для отвязывания **CustomerService** от фактической реализации:

```
type customerStorer interface {
   StoreCustomer(Customer) error
}

type CustomerService struct { storer
  customerStorer
}

func (cs CustomerService) CreateNewCustomer(id
  string) error { customer := Customer{id: id}
  return cs.storer.StoreCustomer(customer)
}
```

Ограничение поведения

```
package main

import ("fmt")

type Config struct {
  dbHost string
}

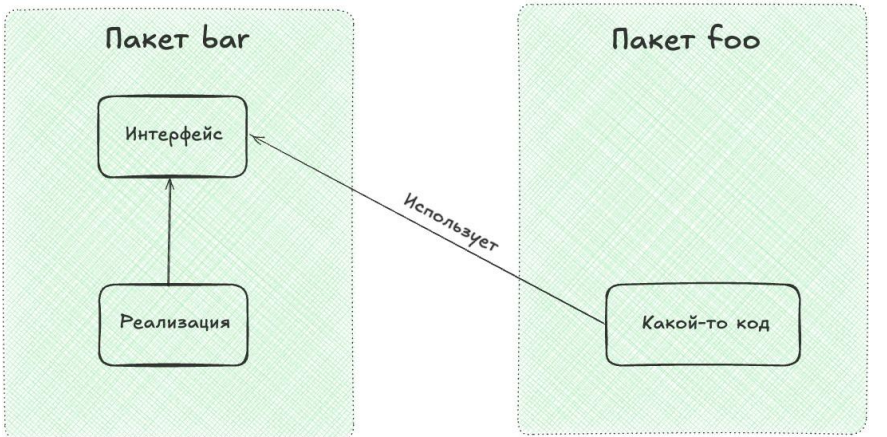
func (c *Config) SetDBHost(host string) {
  c.dbHost = host
}

func (c *Config) DBHost() string {
  return c.dbHost
}
```

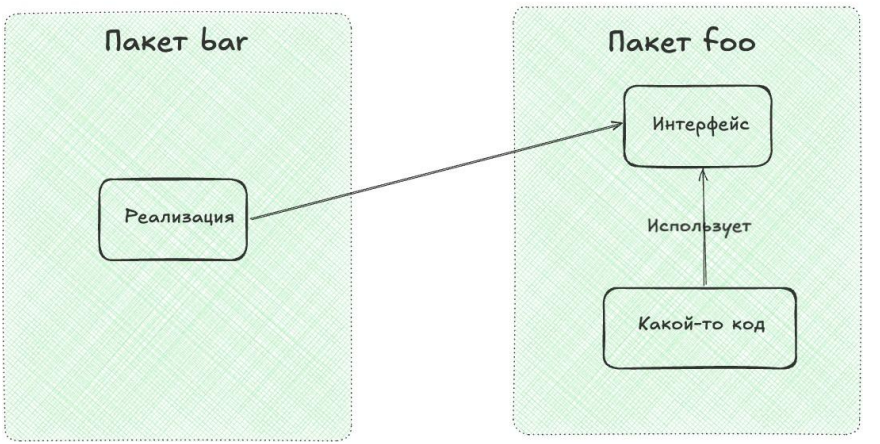
```go
type DbUrlProvider interface {
  DBHost() string
}

func ConnectToDB(reader DbUrlProvider) {
  dbURL := reader.DBHost()
  fmt.Println("Подключение к базе данных по адресу:", dbURL)
}
```

Интерфейс на стороне производителя



Интерфейс на стороне потребителя



Как это выглядит в коде:

```
type orderRepo interface {
  GetOrder(id string) (*Order, error)
}

type Service struct {
  orderRepo orderRepo
}

func (s Service) isExists(orderID string) {
 order, err := s.orderRepo.GetOrder(orderID)
 if err != nil {
    fmt.Println("Ошибка:", err)

    return false
  }
  fmt.Printf("Найден заказ: ID=%s, Total=%.2f\n" , order.ID,
  order.Total)
  return false
```

Следует быть очень осторожными при создании абстракций в коде: их следует обнаруживать, а не создавать.

> Не программируйте интерфейсы, открывайте их.
>
> Роб Пайк
